

Experiment No. 24

Write a Prolog program to implement Monkey Banana Problem.

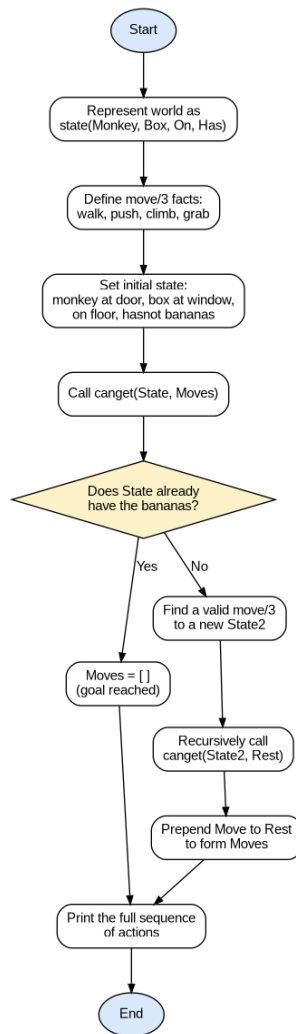
Aim

To write a Prolog program to solve the classic Monkey Banana Problem, where a monkey must walk to a box, push it under the bananas, climb onto it, and grab the bananas using a valid sequence of actions.

Algorithm

1. Start.
2. Represent the state of the world as state(MonkeyPosition, BoxPosition, MonkeyOnFloorOrBox, HasBananasOrNot).
3. Define the move rule for grabbing: move(state(middle, middle, onbox, hasnot), grab, state(middle, middle, onbox, has)).
4. Define the move rule for climbing onto the box when the monkey and box are at the same position.
5. Define the move rule for pushing the box from the monkey's current position to a new position.
6. Define the move rule for walking from one position to another while the box remains where it is.
7. Define the predicate canget(State, Moves) where Moves is the empty list if State already has the bananas.
8. Define the recursive clause of canget(State1, [Move | Moves]) that finds a move from State1 to State2 and recursively solves canget(State2, Moves).
9. Query canget with the initial state, where the monkey is at the door, the box is at the window, and the monkey does not have the bananas.
10. Once the full list of moves is obtained, print each action in the sequence.
11. Display a confirmation message once the bananas have been grabbed.
12. Stop.

Flowchart



Program

```
move(state(middle, middle, onbox, hasnot),  
      grab,  
      state(middle, middle, onbox, has)).
```

```
move(state(P, P, onfloor, H),  
      climb,  
      state(P, P, onbox, H)).
```

```
move(state(P1, P1, onfloor, H),  
      push(P1, P2),  
      state(P2, P2, onfloor, H)).
```

```
move(state(P1, B, onfloor, H),  
      walk(P1, P2),  
      state(P2, B, onfloor, H)).
```

```

canget(State, []) :-
    State = state(_, _, _, has).

canget(State1, [Move | Moves]) :-
    move(State1, Move, State2),
    canget(State2, Moves).

print_moves([]).
print_moves([Move | Moves]) :-
    format(" - ~w~n", [Move]),
    print_moves(Moves).

main :-
    InitialState = state(door, window, onfloor, hasnot),
    canget(InitialState, Moves),
    format("Sequence of actions to reach the bananas:~n"),
    print_moves(Moves),
    format("Monkey has grabbed the bananas!~n").

```

Result

The program was executed successfully and the output was verified as correct.

?- main.

Sequence of actions to reach the bananas:

- walk(door,window)
- push(window,middle)
- climb
- grab

Monkey has grabbed the bananas!

true.

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:

monkeybanana.pl - AI-Lab - Visual Studio Code

EXPLORER

AI-Lab

sum_n.pl

name_dob.pl

student_teacher.pl

planets.pl

hanoi.pl

birdflight.pl

familytree.pl

dietsystem.pl

monkeybanana.pl

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

move(State1 Move, State2)

move(state(middle, middle, onbox, hasnot),

grab,

state(middle, middle, onbox, has)).

move(state(P, P, onfloor, H),

climb,

state(P, P, onbox, H)).

move(state(P1, P1, onfloor, H),

push(P1, P2),

state(P2, P2, onfloor, H)).

move(state(P1, B, onfloor, H),

walk(P1, P2),

state(P2, B, onfloor, H)).

canget(State, []) :-

State = state(_, _, has).

canget(State1, [Move | Moves]) :-

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PS C:/Users/Athavan/AI-Lab> swipl monkeybanana.pl

Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)

?- main.

Sequence of actions to reach the bananas:

- walk(door>window)

- push(window>middle)

- climb

- grab

Monkey has grabbed the bananas!

true.

?-

Prolog

UTF-8 LF Spaces: 4